PHASE 1 / THEORY NOTES

Caching Fundamentals

The concepts every AI Engineer needs before touching a single Redis command — hit/miss, TTL, eviction policies, and read/write patterns.

```
> Caching Concepts (you are here)
    ↓
Redis Fundamentals
    ↓
SWE Use Cases  →  System Design
    ↓
AI/ML Use Cases  →  Agentic AI  →  Production
```

01 — THE CORE IDEA

What Is a Cache, Really?

Every caching concept you'll ever learn is a variation of one idea: keep the data you need **close and fast**, instead of fetching it from somewhere **far and slow** every single time.

> ANALOGY
>
> Think of a library. If you walk to the back warehouse (slow, far away) every time you need a book, you waste time. Instead, you keep the 5 books you use most on your desk (fast, nearby). You only go to the warehouse when a book isn't on your desk.

> WAREHOUSE
>
> **= Database.** Slow, but holds everything, permanently.

> DESK
>
> **= Cache.** Fast, but small and temporary.

Why this matters for you

Every phase after this — Redis commands, rate limiting, LLM response caching, agent memory — is just this same idea applied to a new context. If this mental model is solid, everything downstream becomes "oh, it's the same pattern again," instead of a new thing to memorize.

02 — CORE VOCABULARY

The Words You'll Use Every Day

These five terms show up in every caching conversation, every interview, and every piece of Redis documentation. Get comfortable with them first.

Term	Meaning	Library Analogy
Cache Hit FAST	The data was found in the cache — no need to go to the database.	The book was already on your desk.
Cache Miss SLOW	The data was NOT in the cache — you had to fetch it from the database.	You had to walk to the warehouse.
TTL	"Time To Live" — how long a piece of data is allowed to stay in the cache before it's considered stale.	"This book must be returned in 1 hour."
Expiration	What happens when the TTL runs out — the data becomes invalid and is removed or ignored.	Time's up — the book is taken off your desk.
Eviction	Forcibly removing data because the cache ran out of *space* — not because time ran out.	Your desk is full — you must remove a book to add a new one.

> DON'T MIX THESE UP
>
> **Expiration** is about **time**. **Eviction** is about **space**. This exact distinction is a common interview question.

The one metric that matters in production

```
# Hit Ratio — the real KPI of any cache
Hit Ratio = Cache Hits / (Cache Hits + Cache Misses)

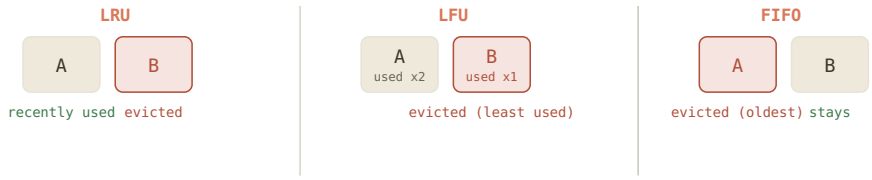
# Example: 90 hits, 10 misses out of 100 requests
Hit Ratio = 90 / 100 = 90%
```

A 90% hit ratio means 90 out of 100 requests were served fast, without touching the database. Production dashboards track this number constantly.

The Desk Is Full — Who Leaves?

A cache has limited space. Once it's full, adding something new means removing something old. The rule that decides *what gets removed* is the eviction policy.

Policy	Rule	When It Shines
LRU Least Recently Used	Remove whatever hasn't been **accessed** for the longest time.	The default choice for most systems. If something hasn't been touched in a while, it probably won't be needed soon either.
LFU Least Frequently Used	Remove whatever has been accessed the **fewest number of times**.	Great when some items are consistently popular (e.g. a trending product) — LFU protects them even if not accessed recently.
FIFO First In, First Out	Remove whatever was **inserted first** — usage is ignored entirely.	Simple and predictable, but can evict something still useful just because it's "old."



Same starting cache, same new item "C" arrives — three different items get evicted.

FIG 1 — HOW EACH EVICTION POLICY REACTS TO THE SAME SITUATION

MEMORY TRICK

LRU = when it was last used (recency) · **LFU** = how often it's used (frequency) · **FIFO** = when it arrived (age)

How Reads & Writes Flow

Knowing cache exists isn't enough — you need to know exactly *where* data flows on a read, and *where* it flows on a write. This is what interviewers and system design docs actually care about.

Read Pattern: Cache-Aside (Lazy Loading)

The most common pattern by far. The application itself is responsible for checking the cache and filling it on a miss.

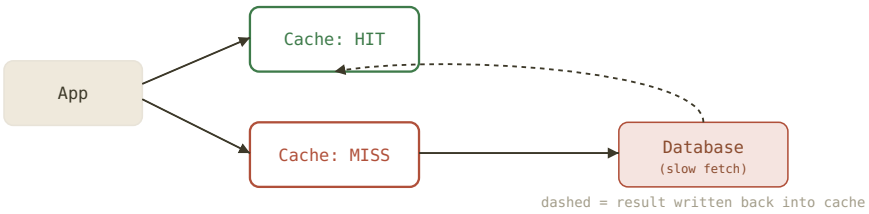


FIG 2 — CACHE-ASIDE READ FLOW

On a **hit**, the database is never touched. On a **miss**, the app fetches from the database, then stores the result in cache for next time.

Write Patterns

Pattern	How It Works	Trade-off
Write-Through	Every write goes to the cache **and** the database at the same time.	✓ Always consistent · ✗ Slightly slower writes
Write-Back (Write-Behind)	Write goes to the cache only; the database is updated later, in a batch.	✓ Very fast writes · ✗ Data can be lost if the cache crashes before the flush
Write-Around	Write goes straight to the database, bypassing the cache entirely.	✓ Keeps the cache clean of rarely-read data · ✗ First read right after a write will be a miss

ONE-LINE SUMMARY

Write-Through = safe but slower. **Write-Back** = fast but risky. **Write-Around** = for data you write often but rarely read right away (e.g. logs).

05 — BEFORE YOU MOVE ON

Phase 1 Checklist

You're ready for Phase 2 (Redis Fundamentals) once you can confidently answer these without looking back:

- [ ] What is a cache hit vs. a cache miss, and how do you calculate hit ratio?
- [ ] What is the real difference between expiration and eviction?
- [ ] Given the same situation, why would LRU, LFU, and FIFO each evict a different item?
- [ ] What happens on a read in the Cache-Aside pattern, step by step?
- [ ] Which write pattern would you choose for: a user profile? a view counter? an audit log? — and why?

> **WHAT'S NEXT**
>
> Phase 2 takes every one of these concepts and rebuilds them using real Redis — the same ideas, just backed by an actual in-memory data store instead of a Python dictionary.